

Introduction to Artificial Intelligence

Lecture 1: Introduction / Course content

September 4, 2025



Self-introduction

- **Name:** Ryan Zhang
- **Position:** Assistant professor at Northeastern University since 2020
- **Research area:** Machine learning and computer science, in particular statistical foundations, algorithms, and optimization
 - **Lab:** <https://virtuosoresearch.github.io/>
 - **GitHub:** <https://github.com/VirtuosoResearch>
 - **Background:** PhD in computer science from Stanford University, postdoc in statistics and data science from the University of Pennsylvania
- **Excited to teach this class!**



Course introduction

- **This semester: An introductory class on artificial intelligence**
- **Intelligence:** For thousands of years, we have tried to understand *how we think and act*. How does our brain perceive, understand, predict, and survive in a world larger and more complicated than the brain?
- **Artificial intelligence (AI):** Not just understanding but also *building* intelligent entities---machines that can compute how to act effectively and safely in new environments



Why learning about AI?

- **Economy:** In 2024, U.S. private AI investment grew to \$109.1 billion
 - 12 times China's \$9.3 billion and 24 times the U.K.'s \$4.5 billion
 - AI business usage is also accelerating: 78% of organizations reported using AI in 2024, up from 55% the year before
 - Globally, corporate AI investment reached \$252.3 billion in 2024, with private investment climbing 44.5% and mergers and acquisitions up 12.1% from the previous year
 - Generative AI usage in at least one business function more than doubled—from 33% in 2023 to 71% last year
- **Reference:** https://hai.stanford.edu/assets/files/hai_ai_index_report_2025.pdf



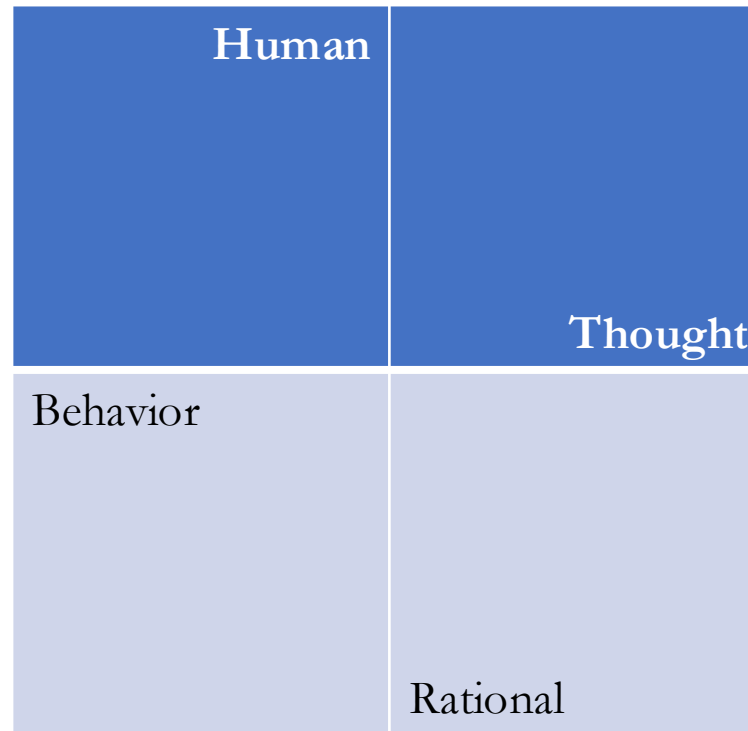
Why learning about AI?

- **Science and technology:** AI's growing importance is reflected in major scientific awards. Two Nobel Prizes recognized work that led to deep learning (physics) and to its application to protein folding (chemistry), while the Turing Award honored groundbreaking contributions to RL
 - **Complex reasoning remains a challenge:** AI models excel at tasks like IMO but still struggle with complex reasoning. They often fail to reliably solve logic tasks even when provably correct solutions exist, limiting their effectiveness in high-stakes settings where precision is critical
- **The intellectual frontier of AI are wide open:** Unlike other fields such as physics, AI still has many openings for full-time masterminds



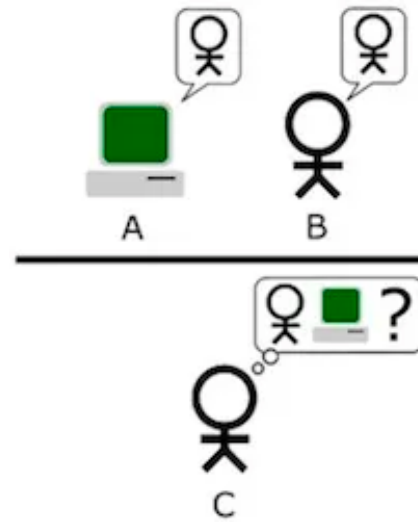
What is AI?

- Desiderata:
 - Intelligence in terms of fidelity to human performance (also known as “rationality”)
 - Capable of internal thought processes and reasoning



What is AI? The Turing test approach

- **“Can a machine think?”** If a computer can fool a human into thinking he or she is conversing with another human rather than a machine, then the computer can be said to be a true example of artificial intelligence



- **Required skills:** natural language processing, knowledge representation, automated reasoning, and machine learning

The rational agent approach

- **Agents:** operate autonomously, perceive their environment, persist over a prolonged time period, adapt to a change, create and pursue goals
 - **Self-driving cars:** sense complex scenes in real time, and make safe driving decisions
 - **Virtual LLM assistant:** understand rich subtleties of natural language, infer use intent, and generate responses



Complex real-world problems

- Bridging the gap



```
Files
  master
  Go to file
  task/hessian
  task/hessian
  __init__.py
  data.py
  datamodels.py
  hessian.py
  plot.py
  solver.py
  svgo.py
  utils.py
  .gitignore
  README.md
  pyproject.toml

Taskmodeling-and-Hessians / task/hessian / hessian.py
Code 1486 lines (1155 loc) · 61.7 KB
29 class Hessian_Calculator():
30     def __init__(self, model, dataloader, loss_fn, noise_vector=None, device='cpu'):
31         self.model = model
32         self.dataloader = dataloader
33         self.loss_fn = loss_fn
34         self.noise_vector = noise_vector
35         self.device = device
36         self.total_params = sum(p.numel() for p in self.model.parameters() if p.requires_grad)
37
38     # Utils
39
40     def evaluate_loss(self, model, dataloader, loss_fn, device):
41         """compute the average loss over a dataloader."""
42         model.eval()
43         total_loss = 0.0
44         total_samples = 0
45         with torch.no_grad():
46             for batch in dataloader:
47                 data, target, batch_size = self.load_batch_func(batch, device)
48                 output = model(data)
49                 loss = loss_fn(output, target)
50                 total_loss += loss.item() * batch_size
51                 total_samples += batch_size
52         return total_loss / total_samples
53
54     def evaluate_hutch(self, model, dataloader, loss_fn, noise_vector=None, device='cpu'):
55         """compute the average hutch over a dataloader."""
56         model.train()
57         total_loss = 0.0
58         total_hutch = 0.0
59         total_max_eig = 0.0
60         count = 0
61         with sdpa_kernel([SDPBackend.MATH]):
62             for batch in dataloader:
63                 data, target, batch_size = self.load_batch_func(batch, device)
64                 output = model(data)
65                 loss = loss_fn(output, target)
66                 total_loss += loss.item() * batch_size
67                 count += batch_size
68
69         # Generate a noise vector matching the flattened parameters.
70         if noise_vector is None:
71             flat_params = torch.cat([p.detach().view(-1) for p in model.parameters()])
72             noise_vector = torch.randn_like(flat_params)
73
74         # Hutchinson estimators:  $\nabla^2 \ell \approx \frac{1}{n} \sum_{i=1}^n \nabla \ell \nabla \ell^T v_i$ 
75         hessian_quad = self.hessian_quadratic_form(model, loss, noise_vector)
76         hessian_quad = hessian_quad.item()
77         total_hutch += hessian_quad * batch_size
78
79         avg_loss = total_loss / count if count > 0 else 0.0
80         avg_hutch = total_hutch / count if count > 0 else 0.0
81         return avg_loss, avg_hutch
82
83     def evaluate_max_eigenvalue(self, model, dataloader, loss_fn, device='cpu'):
84         """compute the maximum eigenvalue over a dataloader."""
85         model.train()
86         total_max_eig = 0.0
87         count = 0
88         with sdpa_kernel([SDPBackend.MATH]):
89             for batch in dataloader:
90                 data, target, batch_size = self.load_batch_func(batch, device)
91                 output = model(data)
92                 loss = loss_fn(output, target)
```



Syllabus

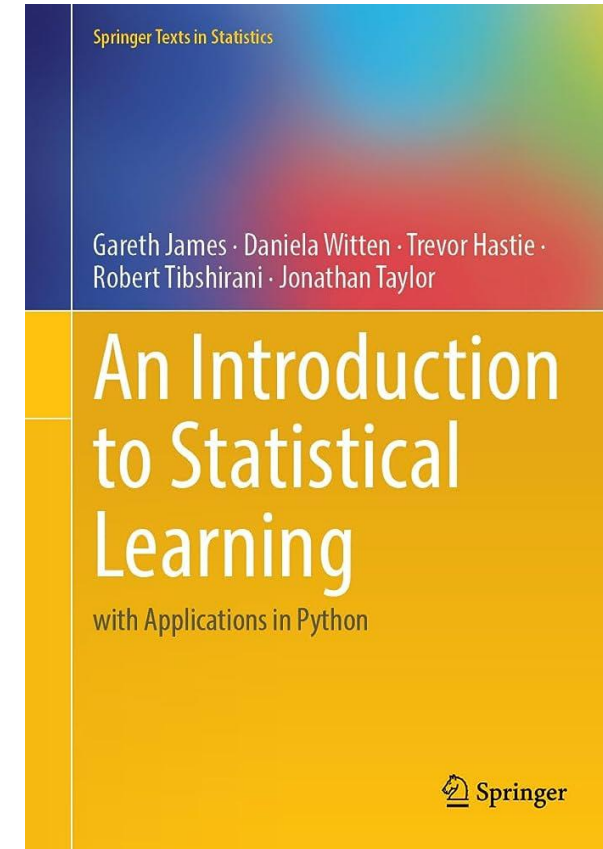
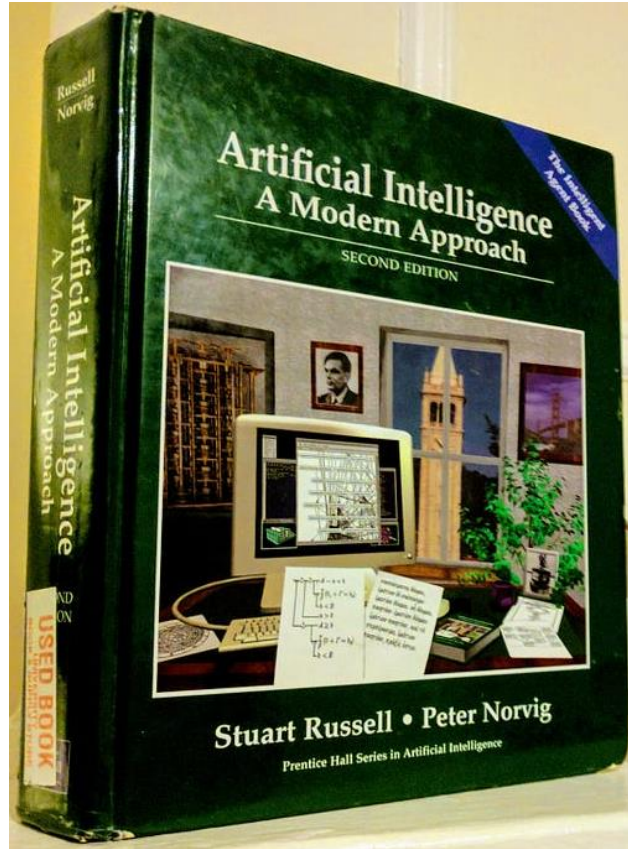
- **Part I:** Machine learning, neural networks, and language modeling
- **Part II:** Intelligent search, games, and reinforcement learning,
- **Part III:** Reasoning, knowledge representation, and uncertainty

- **Prerequisites:** Linear Algebra, Applied Probability, Python
 - **Note:** These are soft requirements in the sense I will use algebra and python at various places. However, you are not required to have taken the corresponding classes beforehand. The TAs and I will make sure to cover up the ground so you have all you need to complete this course

- **Complete syllabus, schedule, and course homepage:** see canvas



Recommended (optional) textbooks



+ Handwritten notes (from me and TAs)



Course information

- **Instructor:** Ryan Zhang
- **Location:** WVG 108
- **Time:** Mondays and Thursdays from 11:45 AM to 1:25 PM



Coursework and grading policy

- Homework 40%
 - 5 - 6 problem sets with a mix of mathematical and coding questions
- Exam 40%
 - Nov 20 – Nov 23, take home, pick 24 hours
- Course project proposal presentation + final project presentation 20%
 - In class, proposal presentation on Oct 20 and Oct 23; final project presentation on Dec 1
 - We will provide a template project around AI agents such as LLMs
 - Another option is to pick a problem you are interested and develop a machine-learning solution to solve it



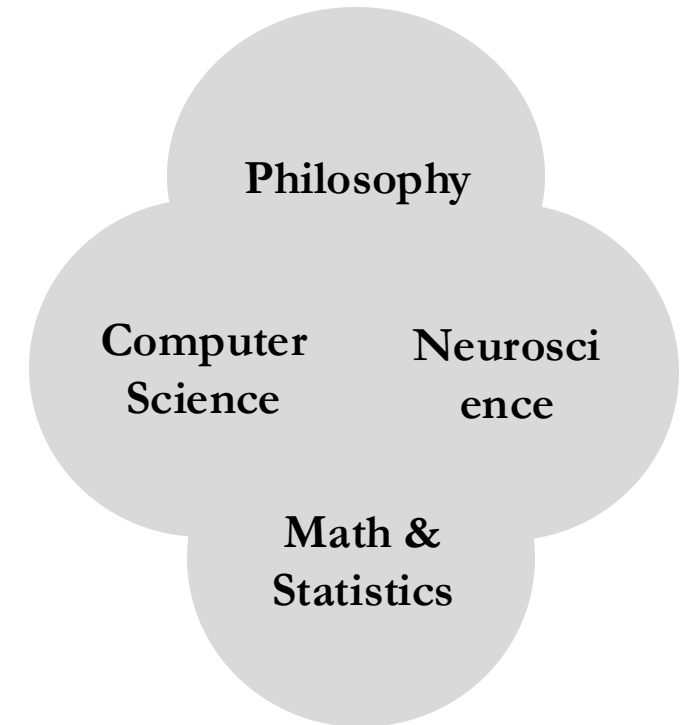
Lecture plan

- **Several examples of machine learning**
- History of AI



Foundations of artificial intelligence

- Back to our question: how to build machines that exhibit intelligent (human-like) behavior?
- Some low-level questions:
 - What are the formal rules to draw valid conclusions?
 - What can be computed?
 - How do we reason with uncertain information?
- Some high-level questions:
 - How do brains process information?
 - How does knowledge lead to action?



Machine learning

- One of the major technical developments that helped fuel the growth of AI is machine learning and deep learning, including large language models, generative models



1st example: Rental cost prediction

- **Example:** Predict the apartment rental cost in Boston



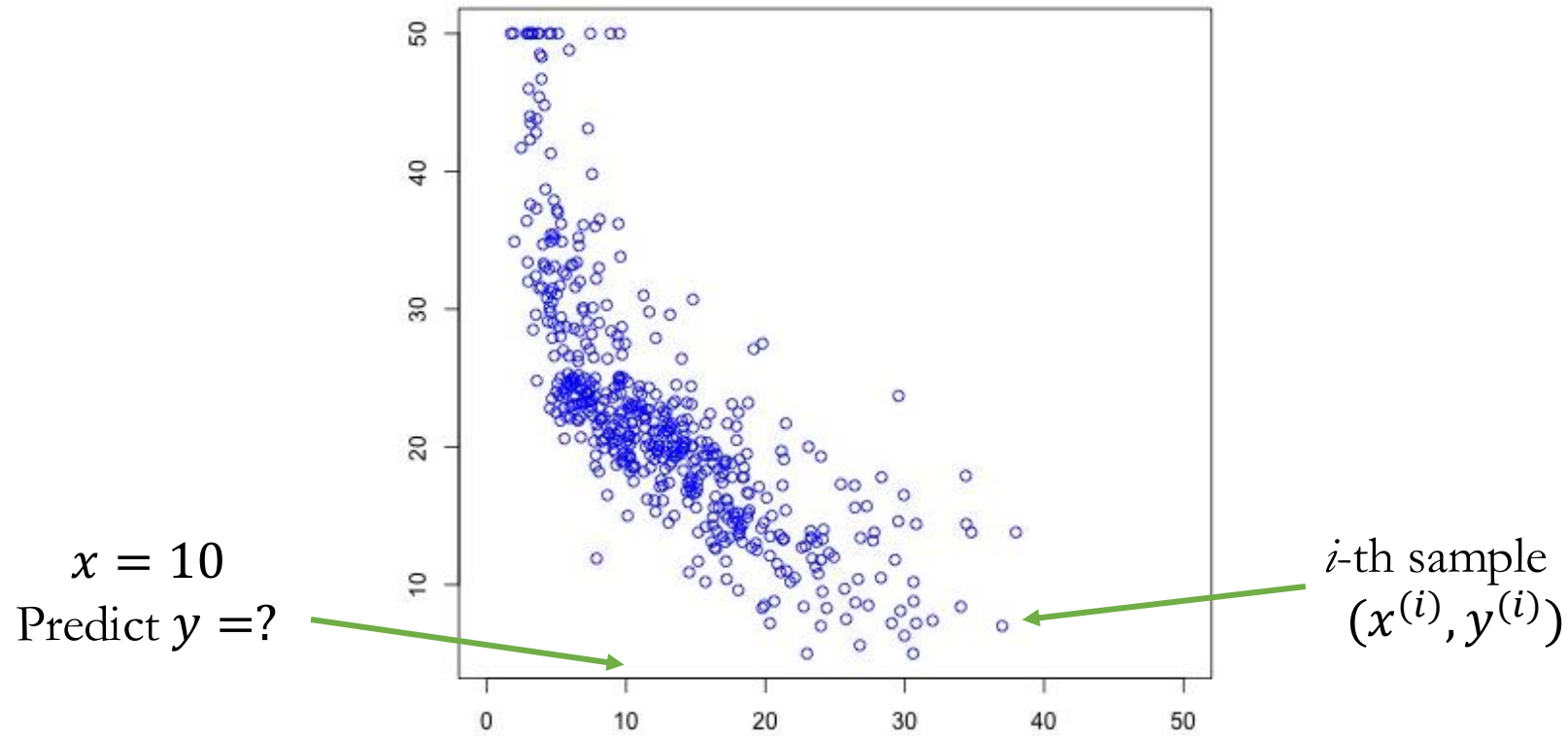
- **Mathematical setup:** $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})$
 - $x^{(i)}$: number of bedrooms, number of bathrooms, number of square feet, zip code, distance to nearest train station, number of groceries nearby...
 - $y^{(i)}$: rental price of the apartment
- **Linear regression:** set up a linear model θ , minimize the mean squared error

$$\hat{L}(\theta) = \frac{1}{n} \sum_{i=1}^n (\theta^\top x^{(i)} - y^{(i)})^2$$



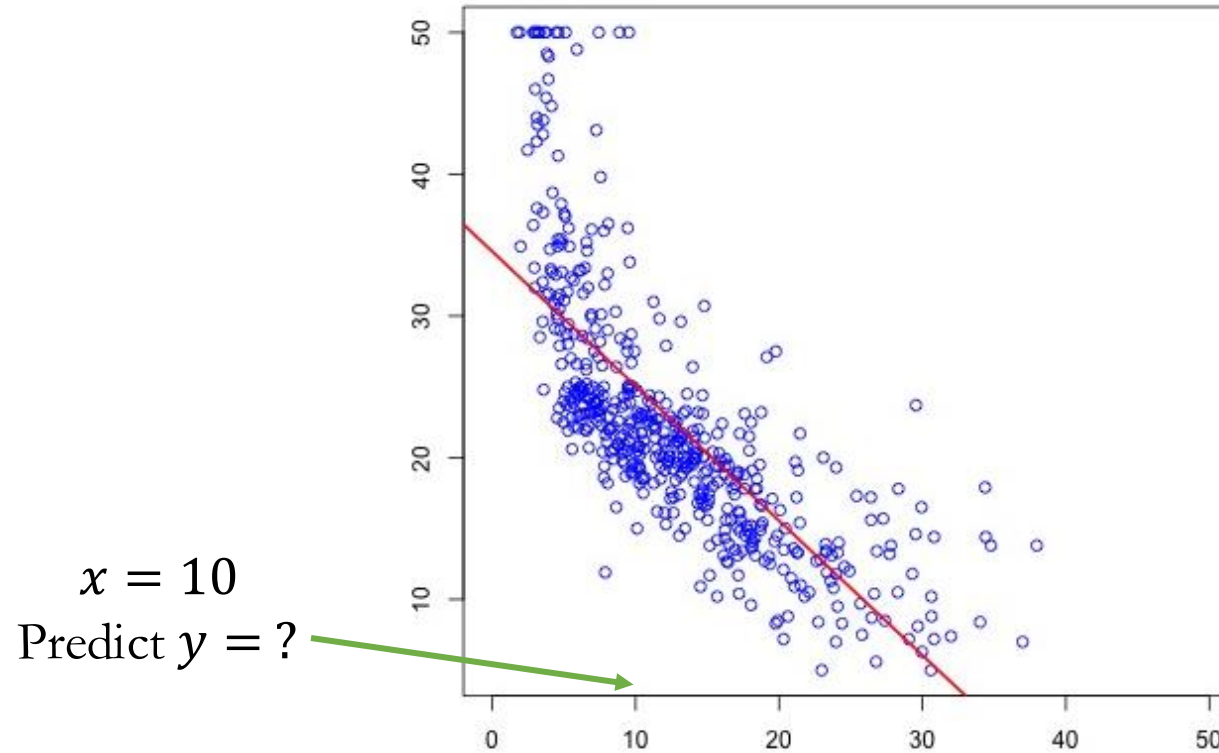
Illustration

- This is a scatter plot of the dataset: LSTAT (income status among population vs. MEDV (median rental value)



Illustration

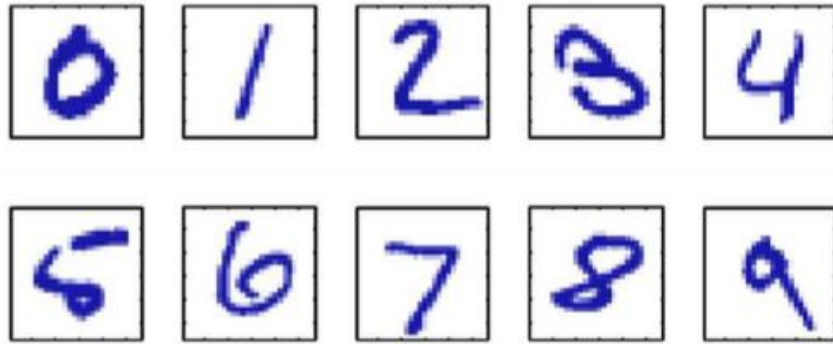
- Fit a line by minimizing the MSE metric



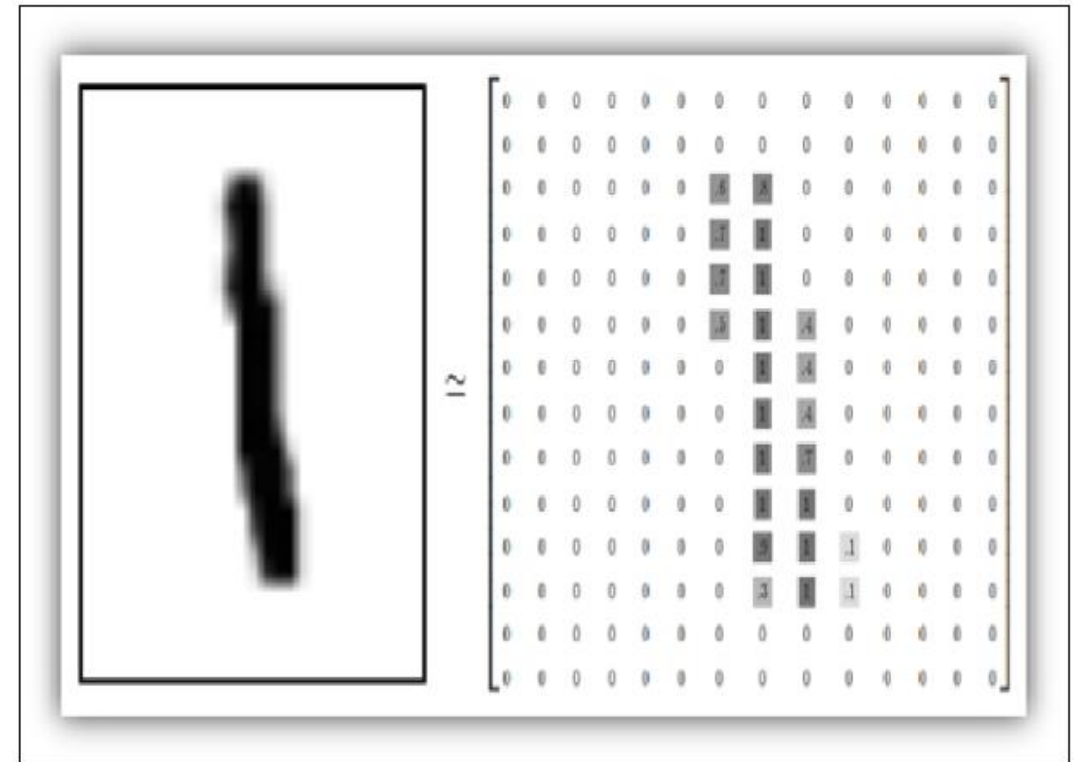
Fit a line to the data

2nd example: Classifying handwritten digits

- Each image is 28 by 28 pixels, corresponding to a 784-dimensional vector

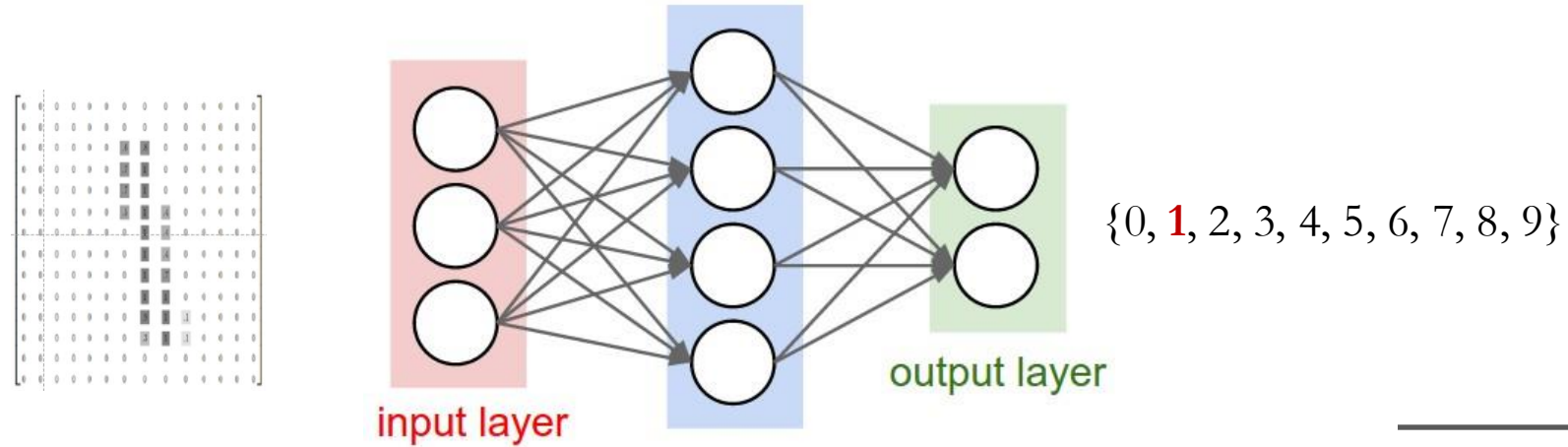


- Preprocessing: First need to map the image to an encoding: Here we'll use a matrix to represent of a black-n-white digit



Feedforward neural networks

- Let's consider the simplest form of neural network models



- Input layer:** Receives the encoding of the handwritten digit 1
- Hidden layer:** Maps the input encoding to a representation through a nonlinear transformation
- Output layer:** Maps the representations to class probabilities

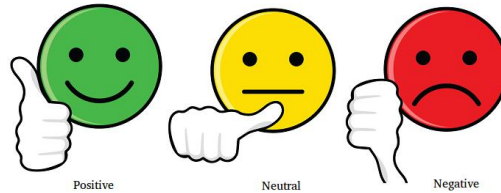
Generalization

- The main conceptual difficulty of learning is that if done properly, the trained model will be able to produce good predictions beyond the set of training examples
- This leap of faith is called **generalization**, either explicitly or implicitly, and is at the heart of any machine learning algorithm
 - This can be formalized using tools from probability and statistical learning theory



3rd example: Sentiment analysis

- Given a list of words: $\{A_1, A_2, \dots, A_n\}$, predict sentiment y which is either positive, neutral, or negative



★★★★★ 5 months ago

Had a great experience at Symphony Sushi! The lobster fried rice and Geisha maki were really delicious—definitely must-tries. Prices are super reasonable for the quality, and the service was fast and friendly. It's also really close to Northeastern, which makes it super convenient. Highly recommend if you're in the area!

Service
Dine in

Meal type
Dinner

Food: 5

Service: 5

Atmosphere: 5



- Choose the most likely hypothesis given the list of words

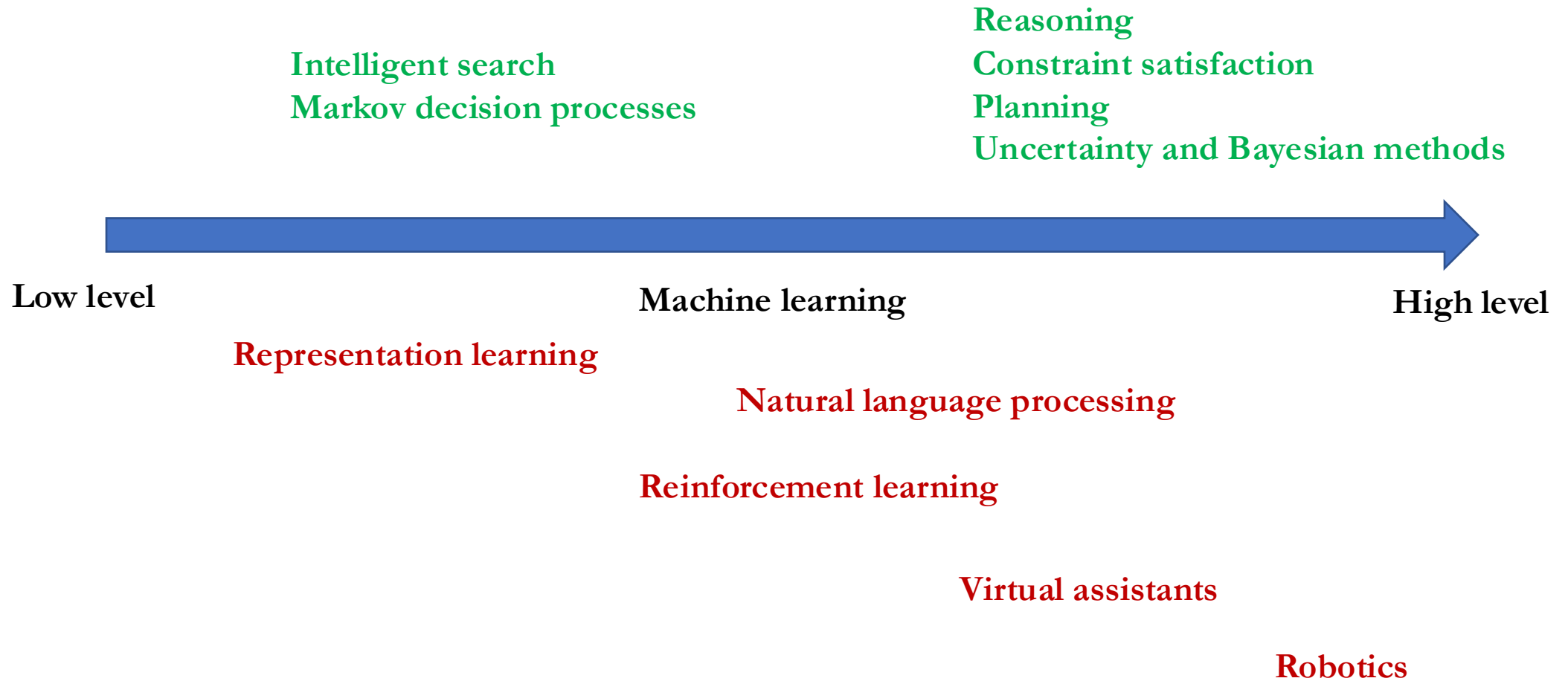
$$\arg \max_y \Pr(A_1, A_2, \dots, A_n | y) = \frac{\Pr(A_1, A_2, \dots, A_n, y)}{\Pr(y)}$$

- Naïve Bayes assumes conditional independence

$$\Pr(A_1, A_2, \dots, A_n | y) \approx \Pr(A_1 | y) \cdot \Pr(A_2 | y) \cdot \dots \cdot \Pr(A_n | y) = \frac{\Pr(A_1, y)}{\Pr(y)} \cdot \dots \cdot \frac{\Pr(A_n, y)}{\Pr(y)}$$



Course plan



Mid break

- Complete survey!! We'll synthesize the survey results and share in the next lecture

<https://forms.gle/b4264df8TajGTfnw5>



Lecture plan

- Several examples of machine learning
- **History of AI**



History of AI

- The inception of artificial intelligence (1943 – 1956)
 - Alan Turing, “Computing Machinery and Intelligence”
 - The imitation game
- John McCarthy (Dartmouth)
 - 1956, summer workshop that includes legendary thinkers including Allen Newell, Herbert Simon, Arthur Samuel
- Newell and Simon introduced a mathematical theorem-proving system called Logic Theorist (ST), capable of proving most of the theorems in Chapter 2 of Russell and Whitehead’s *Principia Mathematica*



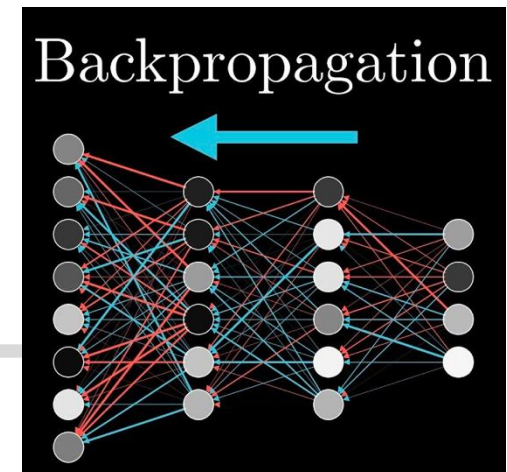
History of AI

- Early enthusiasm, great expectations (1952 – 1969)
 - *Machines will be capable, within twenty years, of doing any work a man can do.* — Herbert Simon
 - *Within 10 years the problems of artificial intelligence will be substantially solved.* —Marvin Minsky
 - *I visualize a time when we will be to robots what dogs are to humans, and I'm rooting for the machines.* —Claude Shannon
- Then come a dose of reality (1966 – 1973)
 - Problems: **limited computation** (search space outpace hardware), **limited information** (complexity of AI)



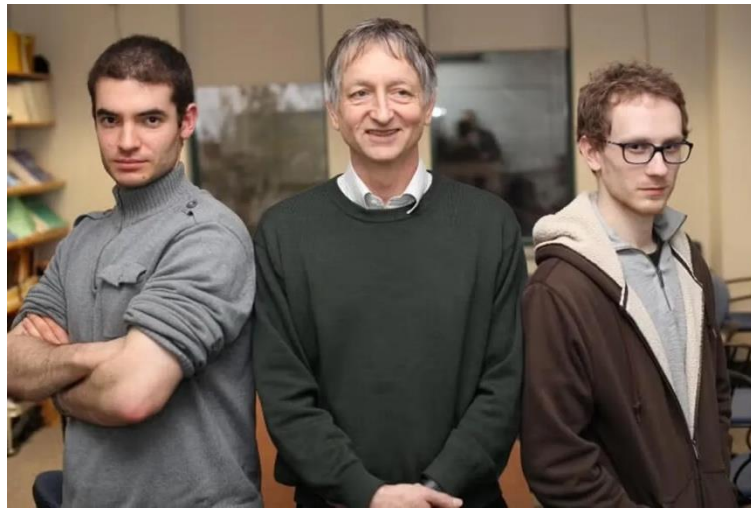
History of AI

- Expert systems, the return of neural networks, probabilistic reasoning, and machine learning
 - 1969-1986: Expert systems elicit specific domain knowledge from experts in form of rules
 - 1986-present: Neural networks become the workhorse of modern machine learning, powered by the back-propagation learning algorithm
 - David McAllester (1998): AI and computer science can benefit from each other
 - Judea Pearl (1988): Introduce Bayesian networks to tackle probabilistic reasoning



Deep learning (2011 – present)

- 2012: AlexNet obtains huge gains in computer vision and object recognition; transformed computer vision
 - **Geoffrey Hinton** (U-Toronto) and his students Ilya Sutskever and Alex Krizhevsky



- 2016: AlphaGo uses deep reinforcement learning, defeat world champion Lee Sedol in the Go game

GPT models

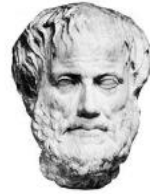
- 2023: **General pretrained transformer (GPT)** models such as GPT-3, Claude, Gemini begin to impact the world
- **Multitasking:** Capable of performing many different tasks simultaneously
- **Adaptivity:** Fine-tuning, in-context learning



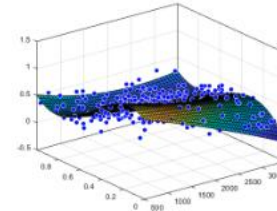
A conflux of ideas and intellectual traditions



neural AI



symbolic AI



statistical AI

- This concludes our tour of the three stories that make up what AI is today
 - Symbolic AI took a top-down approach and failed to fulfill its original promise. But it offered a vision and did build impressive artifacts for ambitious problems like QA and dialogue systems
 - Neural AI is bottom-up, starting with simple tasks. It offered a class of models, deep networks, with today's data and computing resources, capable of conquering ambitious problems
 - Statistical AI foremost offers mathematical rigor and clarity. For example, we define an objective function separate from the optimization algorithm, or have a language to talk about model complexity in learning



Expected outcomes

- You will learn the coolest AI technology, tools during the semester, and grasp the key concept for understanding their working
- During the lectures, my goal is to help you understand the principles of how these methods work, when and how we should use which methods
- **Feel free to ask questions!**



Expected outcomes

- The homework is designed to help you refresh course materials and better understand them
 - Both mathematical and coding questions are involved
 - We encourage you to form groups to complete the homework
 - Ample office hour sessions focused on debugging your coding skills
- Learn how to develop and apply state-of-the-art LLM techniques to downstream tasks
- Learn about prompt tuning, optimization, and AI evaluation



Announcements

- **Office hours:** my email is ho.zhang@northeastern.edu
 - Wednesdays 4 PM to 5 PM, 177 Huntington Ave, 2211. Note: Email me or any one of TAs first to confirm arrival
 - WVH or Zoom (by email)
- **Piazza**
 - Sign up: <https://piazza.com/northeastern/fall2025/cs410013591202610>
 - Access code: mikiuv9zwye
- **Homework 1** will be released later today, due by September 16
 - Submit on gradescope



Reading materials

- Chapter 1, “AI: A modern Approach,” Russell and Norvig
- McKinsey survey: <https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/superagency-in-the-workplace-empowering-people-to-unlock-ais-full-potential-at-work>
- Wikipedia article: https://en.wikipedia.org/wiki/History_of_artificial_intelligence

